

Monitoreo de un clúster AWS EMR con Prometheus y Grafana

Álvaro Prado Peña

INDICE

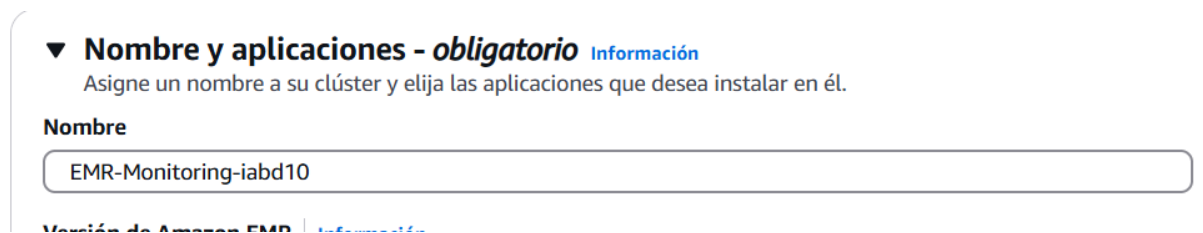
Parte 1: Configuración del clúster AWS EMR	1
Parte 2: Configuración de JMX Exporter	5
Parte 3: Despliegue de Prometheus y Grafana	7
Parte 4: Visualización de métricas en Grafana	13
Preguntas para reflexión	16
1. ¿Qué métricas consideras más importantes para monitorear en un clúster EMR? ¿Por qué?	16
2. ¿Cómo podrías mejorar la configuración de JMX Exporter para recopilar métricas más específicas?	17
3. ¿Qué ventajas tiene usar Prometheus y Grafana frente a otras herramientas de monitoreo?	17

Parte 1: Configuración del clúster AWS EMR

1. Crear un clúster EMR:

o Accede a la consola de AWS y crea un clúster EMR con las siguientes características:

- Nombre del clúster: EMR-Monitoring-iabdXX.



▼ **Nombre y aplicaciones - obligatorio** [Información](#)
Asigne un nombre a su clúster y elija las aplicaciones que desea instalar en él.

Nombre

EMR-Monitoring-iabd10

- Aplicaciones: Hadoop, Spark y Hive.

Una versión contiene un conjunto de aplicaciones que se puede instalar en el clúster.

emr-7.0.0 ▼

Paquete de aplicaciones

Spark Interactive 	Core Hadoop 	Flink 	HBase 	Presto 	Trino 	Custom 
---	---	--	--	--	--	---

☐ AmazonCloudWatchAgent 1.300031.1	☐ Flink 1.18.0	☐ HBase 2.4.17
☐ HCatalog 3.1.3	☒ Hadoop 3.3.6	☒ Hive 3.1.3
☐ Hue 4.11.0	☐ JupyterEnterpriseGateway 2.6.0	☐ JupyterHub 1.5.0
☐ Livy 0.7.1	☐ MXNet 1.9.1	☐ Oozie 5.2.1
☐ Phoenix 5.1.3	☐ Pig 0.17.0	☐ Presto 0.283
☒ Spark 3.5.0	☐ Sqoop 1.4.7	☐ TensorFlow 2.11.0
☐ Tez 0.10.2	☐ Trino 426	☐ Zeppelin 0.10.1
☐ ZooKeeper 3.5.10		

- **Número de instancias: 1 nodo maestro y 2 nodos core.**

Configuración de aprovisionamiento

Establezca el tamaño del principal grupo de instancias. Amazon EMR intenta aprovisionar esta capacidad al lanzar el clúster.

Nombre	Tipo de instancia	Tamaño de instancia(s)	Utilizar la opción de compra de spot
Central	m4.large	3	☐

Grupos de instancias del cluster

Grupos de instancias uniformes

Principal

Elegir tipo de instancia de EC2

m4.large

2 vCore 8 GiB memoria

Únicamente EBS almacenamiento

Precio bajo demanda: -

Precio de spot más bajo: -

Acciones ▼

☐ Utilice la alta disponibilidad

Lance un clúster más resiliente y de alta disponibilidad con tres nodos principales en instancias bajo demanda. Esta configuración se aplica durante toda la vida útil del clúster. [Más información](#)

► Configuración de nodo - *opcional*

Central

Elegir tipo de instancia de EC2

m4.large

2 vCore 8 GiB memoria

Únicamente EBS almacenamiento

Precio bajo demanda: -

Precio de spot más bajo: -

Acciones ▼

Eliminar grupo de instancias

► Configuración de nodo - *opcional*

▼ Terminación del clúster y reemplazo de nodos [Información](#)

Elija la configuración de terminación y proteja su clúster contra un apagado accidental.

Opción de terminación

- ☐ Terminar manualmente el clúster
- ☐ Terminar automáticamente el clúster después de que finalice el último paso
- ☒ Terminar el clúster después del tiempo de inactividad (recomendado)

Tiempo de inactividad

Ingresa el tiempo hasta que el clúster termine.

0 días ▼

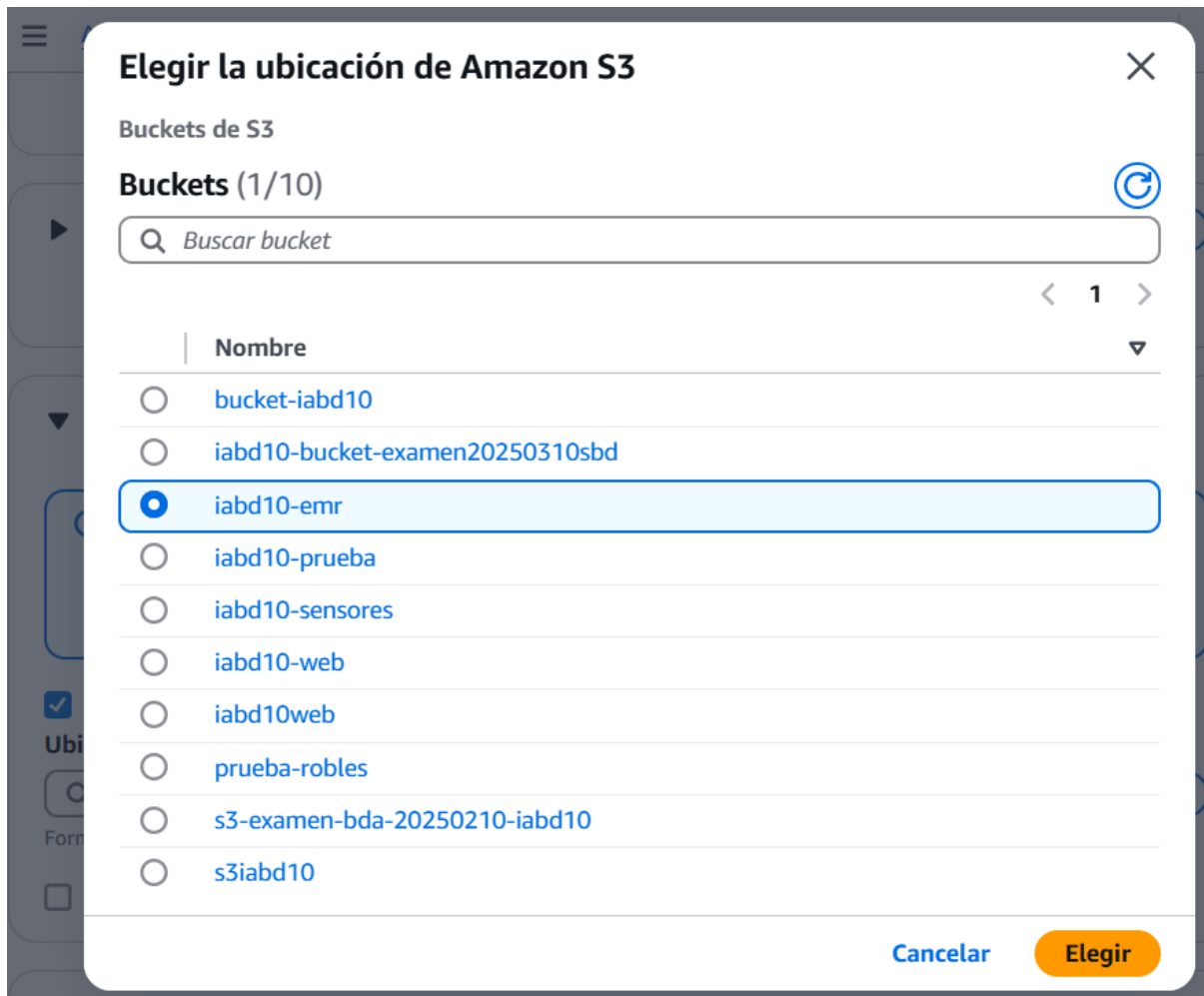
04:00:00

Elija una hora mayor a 1 minuto (00:01:00) y menor a 7 días. La hora está en formato hh:mm:ss (24 horas).

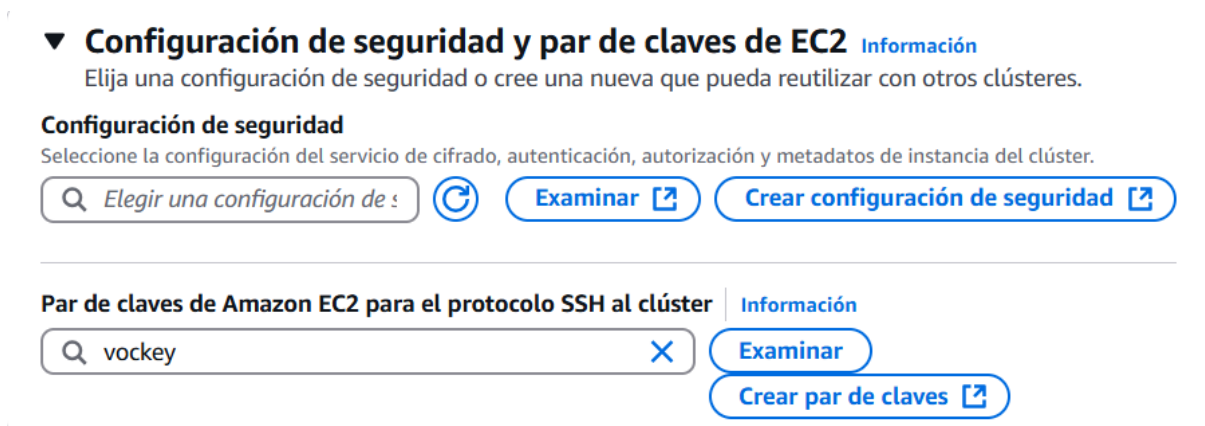
☐ Use la protección contra la terminación

Protege al clúster para evitar una terminación accidental. Si está activada, deberá primero desactivar la protección para terminar el clúster. Recomendamos activar la protección frente a terminaciones para los clústeres de larga duración.

Ponemos que el cluster termine al de 4 horas



Elegimos el bucket ya creado



En la configuracion de seguridad y par de claves de EC2 seleccionamos de la vockey

Información del clúster

ARN del clúster
 `arn:aws:elasticmapreduce:us-east-1:142953221292:cluster/j-26GFG9Q1Q8QSX`

Capacidad

Aplicaciones instaladas
Hadoop 3.3.6, Hive 3.1.3, Spark 3.5.0

Conectarse al nodo principal mediante SSH

Tiempo transcurrido
2 minutos, 20 segundos

Grupos de instancias (2)

Información

Terminar la instancia

Cambiar tamaño de grupo de instancias

Agregar grupo de instancias de tareas

Con la configuración de grupos de instancias, cada tipo de nodo consta del mismo tipo de instancia y de la misma opción de compra de instancias: bajo demanda o spot.

Buscar instancias por estado

Q

Buscar recursos por ID o tipo, o buscar texto dentro de los resultados cargados

<

1

>

🌐

	Tipo y nombre	ID	Estado	Instancias	Opción de compra y precio	Tamaño de EBS (GiB)	ID de instancia EC2
			Motivo del cambio de estado				
☐	 Principal	ig-3Q38GOEKNXZ5	 En ejecución -	1	Bajo demanda	-	-
☐	 Principal (Central)	ig-335JTYF34RI3D	 En ejecución -	1	Bajo demanda	-	-

Activar Windows

2. Conectar al nodo maestro:

- [illegible]

Parte 2: Configuración de JMX Exporter

1. Instalar JMX Exporter:

Explicar o dar un razonamiento extenso de que es y para que sirve JMX en este entorno o el JMX Exporter en el nodo maestro:

JMX (Java Management Extensions) es una tecnología de Java que permite la monitorización y gestión de aplicaciones en tiempo de ejecución. Básicamente, actúa como una interfaz estándar que expone información interna (métricas, estadísticas,

configuración, etc.) de las aplicaciones Java, como JVMs, servidores de aplicaciones o servicios como Apache Kafka, Cassandra, Jenkins, entre otros.

En un entorno como un clúster en AWS, donde probablemente estamos ejecutando aplicaciones distribuidas y críticas sobre la JVM (por ejemplo, Hadoop, Spark, o Jenkins), JMX es fundamental para:

Obtener métricas en tiempo real de las aplicaciones Java.

Identificar cuellos de botella (uso de CPU, memoria, GC, hilos, etc.).

Ayudar en la observabilidad y mantenimiento proactivo del sistema.

Integrarse con herramientas externas como Prometheus y Grafana para una visualización centralizada y alertas.

¿Qué es JMX Exporter?

El JMX Exporter (también conocido como Prometheus JMX Exporter) es una herramienta que convierte métricas JMX a formato Prometheus, para que estas puedan ser recolectadas y luego visualizadas en Grafana. Funciona como un puente entre las aplicaciones Java (que exponen JMX) y Prometheus.

wget

https://repo1.maven.org/maven2/io/prometheus/jmx/jmx_prometheus_javaagent/0.16.1/jmx_prometheus_javaagent-0.16.1.jar

Descargamos el jmx exporter

```
[ec2-user@ip-172-31-30-186 ~]$ wget https://repo1.maven.org/maven2/io/prometheus/jmx/jmx_prometheus_javaagent/0.16.1/jmx_prometheus_javaagent-0.16.1.jar
--2025-03-24 19:08:18-- https://repo1.maven.org/maven2/io/prometheus/jmx/jmx_prometheus_javaagent/0.16.1/jmx_prometheus_javaagent-0.16.1.jar
Resolving repo1.maven.org (repo1.maven.org)... 146.75.32.209, 2a04:4e42:78::209
Connecting to repo1.maven.org (repo1.maven.org)|146.75.32.209|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 469645 (459K) [application/java-archive]
Saving to: 'jmx_prometheus_javaagent-0.16.1.jar'

jmx_prometheus_javaagent-0.16 100%[=====] 458.64K --.-KB/s in 0.01s
2025-03-24 19:08:18 (36.5 MB/s) - 'jmx_prometheus_javaagent-0.16.1.jar' saved [469645/469645]
```

Creamos el archivo de configuración con **nano config.yml**

```
[ec2-user@ip-172-31-18-12 hadoop]$ sudo nano /home/hadoop/config.yml
```

Y dentro del archivo de configuracion metemos lo siguiente:

```
GNU nano 5.8 /home/hadoop/config.yml
lowercaseOutputName: true
lowercaseOutputLabelNames: true
rules:
  - pattern: ".*"
```

```
[ec2-user@ip-172-31-18-12 ~]$ sudo chown hadoop:hadoop jmx_prometheus_javaagent-0.16.1.jar
[ec2-user@ip-172-31-18-12 ~]$
```

Le damos permisos al jar

```
[ec2-user@ip-172-31-18-12 ~]$ sudo nano /home/hadoop/config.yml
```

Añadimos la siguiente línea dentro del archivo

```
export HADOOP_NAMENODE_OPTS=-javaagent:/opt/jmx_prometheus_javaagent-0.16.1.jar=7070:/opt/config.yml
```

Después reiniciamos el namenode

```
[ec2-user@ip-172-31-18-12 ~]$ sudo systemctl restart hadoop-hdfs-namenode
```

Parte 3: Despliegue de Prometheus y Grafana

1. Crear una instancia EC2 para Prometheus y Grafana:

Nombre

[Agregar etiquetas adicionales](#)

▼ **Imágenes de aplicaciones y sistemas operativos (Imagen de máquina de Amazon)** [Información](#)

Una AMI es una plantilla que contiene la configuración de software (sistema operativo, servidor de aplicaciones y aplicaciones) necesaria para lanzar la instancia. Busque o examine las AMI si no ve lo que busca a continuación.

Recientes Mis AMI **Inicio rápido**

Amazon Linux

macOS

Ubuntu

Windows

Red Hat

SUSE Linux

Debian

Buscar más AMI

Inclusión de AMI de AWS, Marketplace y la comunidad

Imágenes de máquina de Amazon (AMI)

Ubuntu Server 22.04 LTS (HVM), SSD Volume Type

ami-0f9de6e2d2f067fca (64 bits (x86)) / ami-0967e5535761d839e (64 bits (Arm))

Virtualización: hvm Activado para ENA: true Tipo de dispositivo raíz: ebs

Apto para la capa gratuita

Seleccionamos ubuntu

▼ Tipo de instancia [Información](#) | [Obtener asesoramiento](#)

Tipo de instancia

t2.micro

Apto para la capa gratuita

Familia: t2 1 vCPU 1 GiB Memoria Generación actual: true

Bajo demanda Windows base precios: 0.0162 USD por hora

Bajo demanda Ubuntu Pro base precios: 0.0134 USD por hora

Bajo demanda SUSE base precios: 0.0116 USD por hora

Bajo demanda RHEL base precios: 0.026 USD por hora Bajo demanda Linux base precios: 0.0116 USD por hora

☐ Todas las generaciones

[Comparar tipos de instancias](#)

Se aplican costos adicionales a las AMI con software preinstalado

▼ Par de claves (inicio de sesión) [Información](#)

Puede utilizar un par de claves para conectarse de forma segura a la instancia. Asegúrese de que tiene acceso al par de claves seleccionado antes de lanzar la instancia.

Nombre del par de claves - obligatorio

vockey

[Crear un nuevo par de claves](#)

Ponemos en el tipo de instancia el t2.micro y en el par de claves la de vockey

▼ Configuraciones de red [Información](#)

[Editar](#)

Red [Información](#)

vpc-0925aa250c0c2c434

Subred [Información](#)

Sin preferencias (subred predeterminada en cualquier zona de disponibilidad)

Asignar automáticamente la IP pública [Información](#)

Habilitar

Se aplican cargos adicionales cuando no se cumplen los límites del nivel gratuito

Firewall (grupos de seguridad) [Información](#)

Un grupo de seguridad es un conjunto de reglas de firewall que controlan el tráfico de la instancia. Agregue reglas para permitir que un tráfico específico llegue a la instancia.

☒ Crear grupo de seguridad

☐ Seleccionar un grupo de seguridad existente

Crearemos un nuevo grupo de seguridad denominado "launch-wizard-8" con las siguientes reglas:

☒ Permitir el tráfico de SSH desde

Ayuda a establecer conexión con la instancia

Cualquier lugar

0.0.0.0/0

☒ Permitir el tráfico de HTTPS desde Internet

Para configurar un punto de enlace, por ejemplo, al crear un servidor web

☒ Permitir el tráfico de HTTP desde Internet

Para configurar un punto de enlace, por ejemplo, al crear un servidor web

En configuraciones de red permitimos el trafico en SSH, HTTP y HTTPS

Editar reglas de entrada [Información](#)

Las reglas de entrada controlan el tráfico entrante que puede llegar a la instancia.

Reglas de entrada [Información](#)

ID de la regla del grupo de seguridad	Tipo Información	Protocolo Información	Intervalo de puertos Información	Origen Información	Descripción: opcional Información	
sgr-0e3f97aa6e796e689	HTTP	TCP	80	Perso...		Eliminar
sgr-020a343d1cd0da136	SSH	TCP	22	Perso...		Eliminar
sgr-0df5375b6ed47757c	HTTPS	TCP	443	Perso...		Eliminar

[Agregar regla](#)

Aquí están las reglas de entrada

ID de la regla del grupo de seguridad	Tipo	Protocolo	Intervalo de puertos	Origen	Descripción: opcional	
sgr-0e3f97aa6e796e689	HTTP	TCP	80	Perso...	Q	Eliminar
sgr-020a343d1cd0da136	SSH	TCP	22	Perso...	Q	Eliminar
sgr-0df3375b6ed47757c	HTTPS	TCP	443	Perso...	Q	Eliminar
-	TCP personalizado	TCP	9090	Anyw...	Q	Eliminar
-	TCP personalizado	TCP	3000	Perso...	Q	Eliminar

[Agregar regla](#)

Añadimos dos reglas para poner los puertos de prometheus y de grafana

```

ubuntu@ip-172-31-88-81: ~
System information as of Tue Mar 25 18:33:04 UTC 2025

System load:  0.0          Processes:      101
Usage of /:   21.8% of 7.57GB   Users logged in:  0
Memory usage: 20%          IPv4 address for eth0: 172.31.88.81
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-88-81:~$

```

Despues de crearlo le damos a conectar por ssh y mediante el archivo labsuser.pem nos conectamos al ssh

```

ubuntu@ip-172-31-88-81:~$ wget https://github.com/prometheus/prometheus/releases/download/v2.30.3/prometheus-2.30.3.linux-amd64.tar.gz
--2025-03-25 18:33:44-- https://github.com/prometheus/prometheus/releases/download/v2.30.3/prometheus-2.30.3.linux-amd64.tar.gz
Resolving github.com (github.com)... 140.82.112.4
Connecting to github.com (github.com)|140.82.112.4|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://objects.githubusercontent.com/github-production-release-asset-2e65be/6838921/bc9e0970-09b3-4893-a66b-5fb918691ale?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=releaseassetproduction%2F20250325%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20250325T183344Z&X-Amz-Expires=300&X-Amz-Signature=8b44abc1f708ad00d5b75142eff970a015cce802e303ef62f7efeb9ad4b43d54&X-Amz-SignedHeaders=host&response-content-disposition=attachment%3B%20filename%3Dprometheus-2.30.3.linux-amd64.tar.gz&response-content-type=application%2Foctet-stream [following]
--2025-03-25 18:33:44-- https://objects.githubusercontent.com/github-production-release-asset-2e65be/6838921/bc9e0970-09b3-4893-a66b-5fb918691ale?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=releaseassetproduction%2F20250325%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20250325T183344Z&X-Amz-Expires=300&X-Amz-Signature=8b44abc1f708ad00d5b75142eff970a015cce802e303ef62f7efeb9ad4b43d54&X-Amz-SignedHeaders=host&response-content-disposition=attachment%3B%20filename%3Dprometheus-2.30.3.linux-amd64.tar.gz&response-content-type=application%2Foctet-stream
Resolving objects.githubusercontent.com (objects.githubusercontent.com)... 185.199.109.133, 185.199.110.133, 185.199.111.133, ...
Connecting to objects.githubusercontent.com (objects.githubusercontent.com)|185.199.109.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 72638078 (69M) [application/octet-stream]
Saving to: 'prometheus-2.30.3.linux-amd64.tar.gz'

prometheus-2.30.3.linux-amd64 100%[=====] 69.27M 92.6MB/s in 0.7s

2025-03-25 18:33:44 (92.6 MB/s) - 'prometheus-2.30.3.linux-amd64.tar.gz' saved [72638078/72638078]

```

Una vez dentro descargamos el prometheus.

```
ubuntu@ip-172-31-88-81:~$ ls
prometheus-2.30.3.linux-amd64.tar.gz
```

```
ubuntu@ip-172-31-88-81:~$ tar -xzf prometheus-2.30.3.linux-amd64.tar.gz
ubuntu@ip-172-31-88-81:~$
```

Descomprimos el jar.

```
ubuntu@ip-172-31-88-81:~$ cd prometheus-2.30.3.linux-amd64
ubuntu@ip-172-31-88-81:~/prometheus-2.30.3.linux-amd64$
```

Ahora hay que modificar el archivo de configuración para que apunte al clúster EMR.

```
GNU nano 6.2 prometheus.yml *
# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  # The job name is added as a label 'job=<job_name>' to any timeseries scraped by this config.
  - job_name: "prometheus"

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["localhost:9090"]

  - job_name: "emr-namenode"
    static_configs:
      - targets: ["3.80.99.53:7070"]
```

Instalación de Grafana

```
ubuntu@ip-172-31-88-81:~/prometheus-2.30.3.linux-amd64$ sudo apt-get install -y apt-transport-https
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Note, selecting 'apt' instead of 'apt-transport-https'
apt is already the newest version (2.4.13).
apt set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
ubuntu@ip-172-31-88-81:~/prometheus-2.30.3.linux-amd64$ sudo apt-get install -y software-properties-common wget
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
software-properties-common is already the newest version (0.99.22.9).
software-properties-common set to manually installed.
wget is already the newest version (1.21.2-2ubuntu1.1).
wget set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
ubuntu@ip-172-31-88-81:~/prometheus-2.30.3.linux-amd64$ wget -q -O - https://packages.grafana.com/gpg.key | sudo apt-key add-
Usage: apt-key [--keyring file] [command] [arguments]

Manage apt's list of trusted keys

  apt-key add <file>          - add the key contained in <file> ('-' for stdin)
  apt-key del <keyid>         - remove the key <keyid>
  apt-key export <keyid>      - output the key <keyid>
  apt-key exportall           - output all trusted keys
  apt-key update              - update keys using the keyring package
  apt-key net-update          - update keys using the network
  apt-key list                - list keys
  apt-key finger              - list fingerprints
  apt-key adv                 - pass advanced options to gpg (download key)

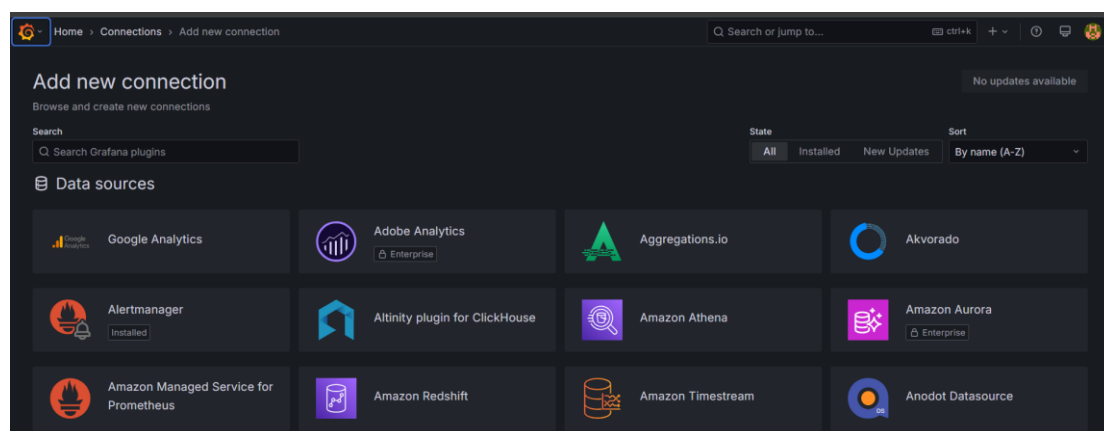
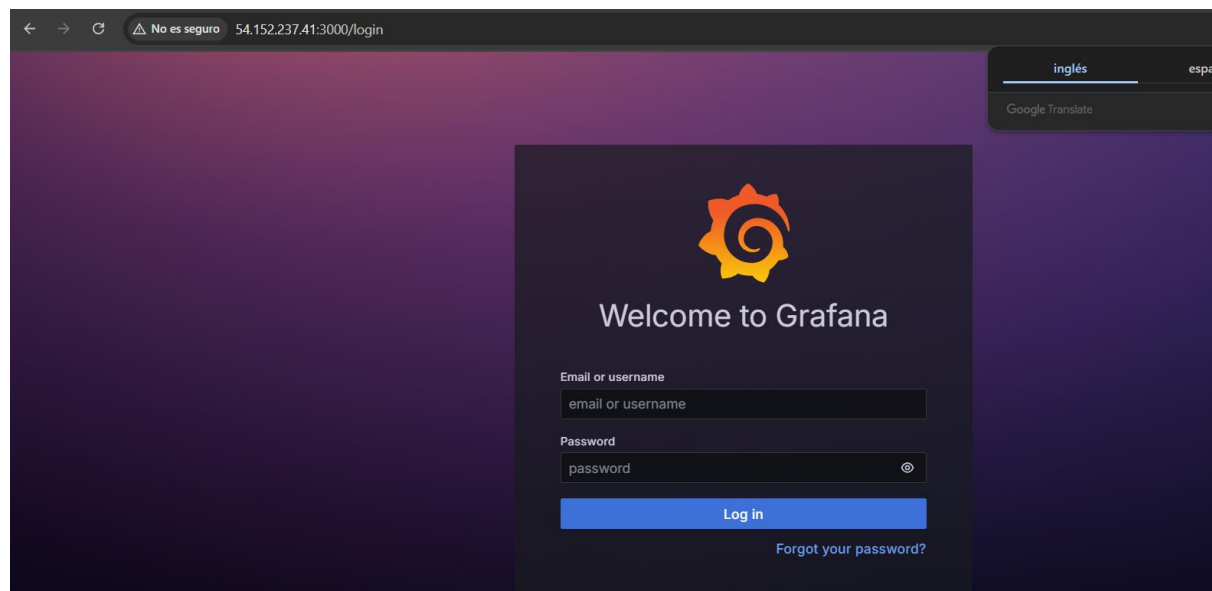
If no specific keyring file is given the command applies to all keyring files.
ubuntu@ip-172-31-88-81:~/prometheus-2.30.3.linux-amd64$
ubuntu@ip-172-31-88-81:~/prometheus-2.30.3.linux-amd64$ echo "deb https://packages.grafana.com/oss/deb stable main" | sudo tee -a /etc/apt/sources.list.d/grafana.list
deb https://packages.grafana.com/oss/deb stable main
ubuntu@ip-172-31-88-81:~/prometheus-2.30.3.linux-amd64$
```

```
ubuntu@ip-172-31-88-81:~/prometheus-2.30.3.linux-amd64$ sudo apt-get update
ubuntu@ip-172-31-88-81:~/prometheus-2.30.3.linux-amd64$ sudo apt-get install grafana
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  musl
The following NEW packages will be installed:
  grafana musl
0 upgraded, 2 newly installed, 0 to remove and 13 not upgraded.
Need to get 140 MB of archives.
After this operation, 537 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy/universe amd64 musl amd64 1.2.2-4 [407 kB]
Get:2 https://packages.grafana.com/oss/deb stable/main amd64 grafana amd64 11.5.3 [140 MB]
35% [2 grafana 43.8 MB/140 MB 31%]
```

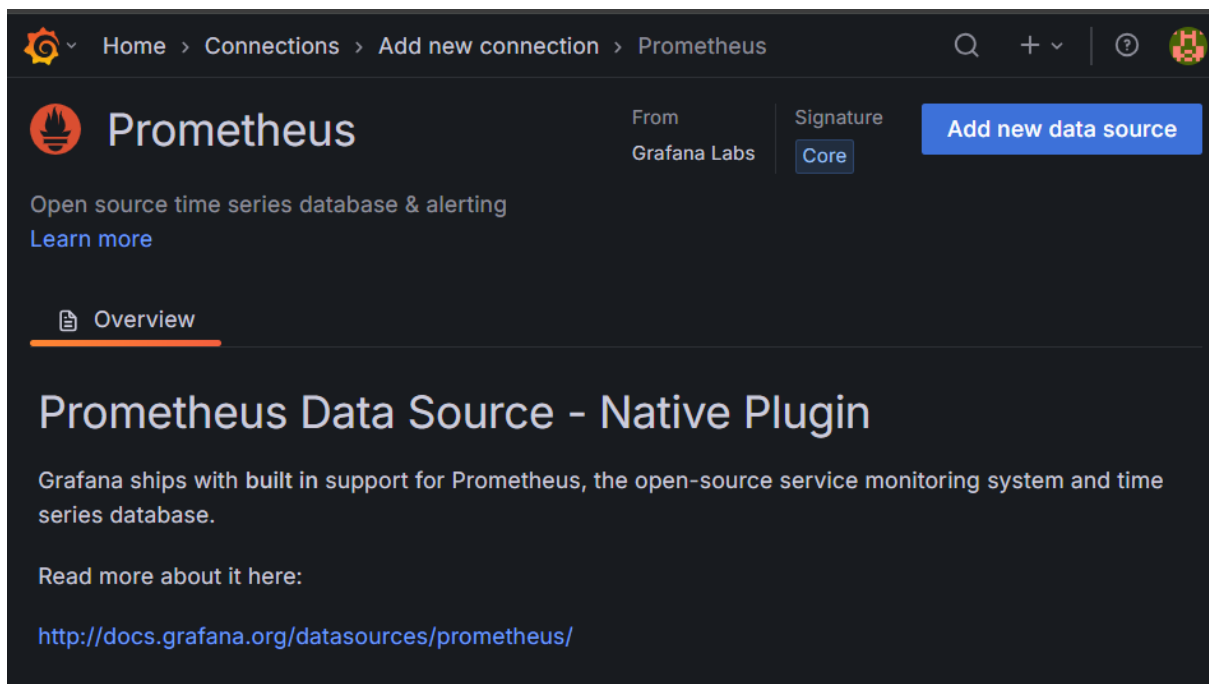
Una vez instalado el grafana lanzamos el servidor de grafana

```
ubuntu@ip-172-31-88-81:~/prometheus-2.30.3.linux-amd64$ sudo systemctl start grafana-server
ubuntu@ip-172-31-88-81:~/prometheus-2.30.3.linux-amd64$ sudo systemctl enable grafana-server
Synchronizing state of grafana-server.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable grafana-server
Created symlink /etc/systemd/system/multi-user.target.wants/grafana-server.service → /lib/systemd/system/grafana-server.service.
ubuntu@ip-172-31-88-81:~/prometheus-2.30.3.linux-amd64$
```

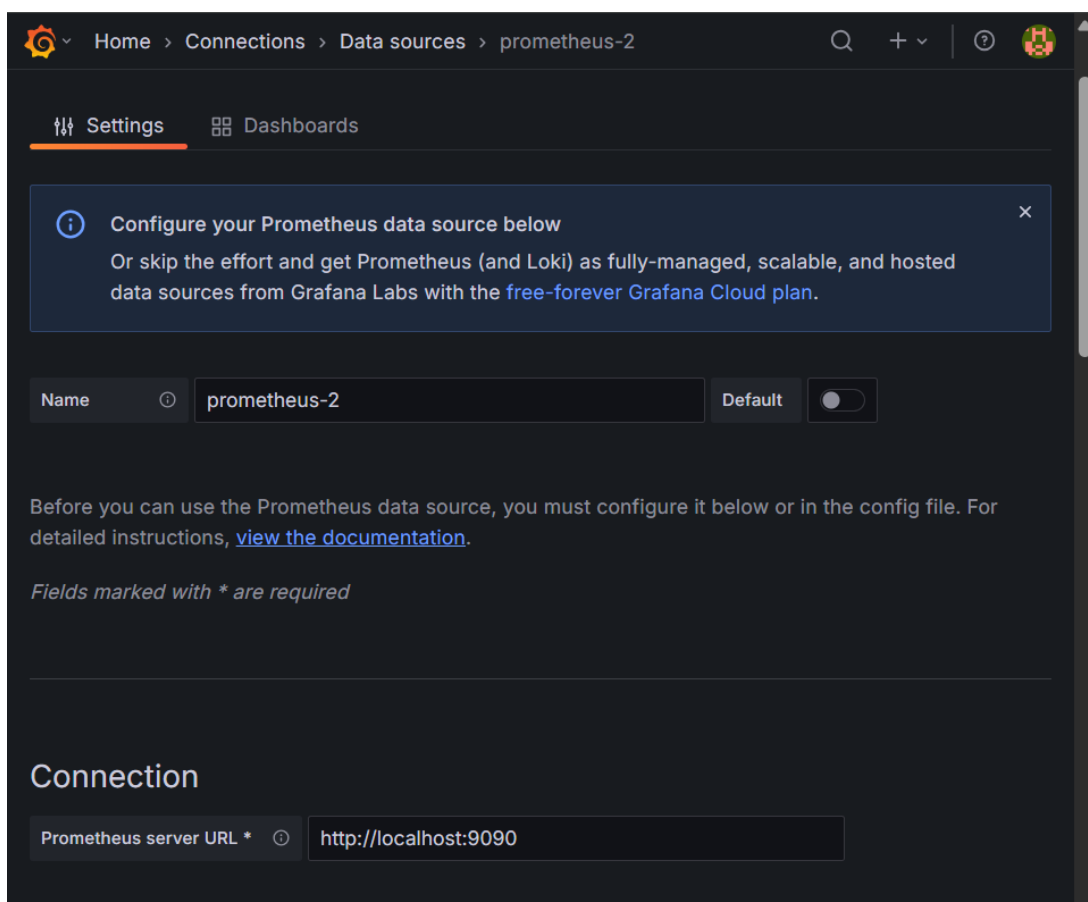
Una vez en Grafana iniciamos sesion con admin admin



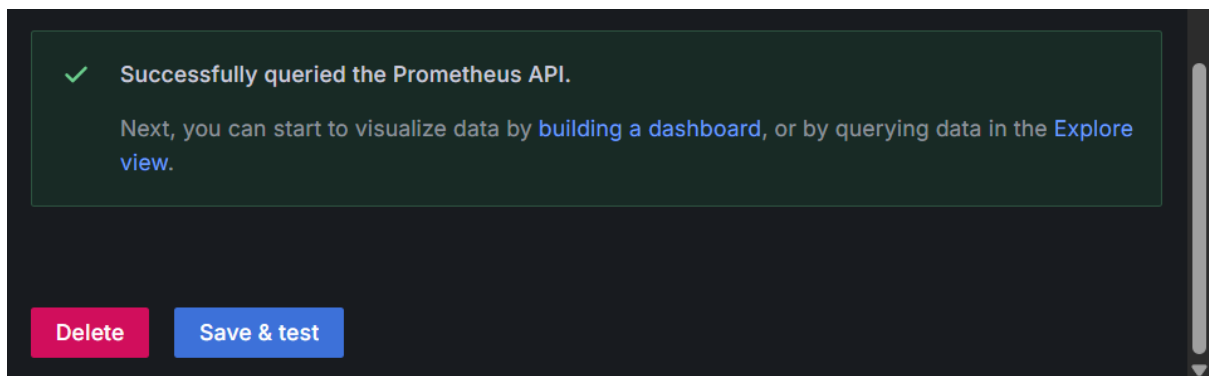
Aqui seleccionamos connectios y add new connection



Vale ahora le daremos a add new data source

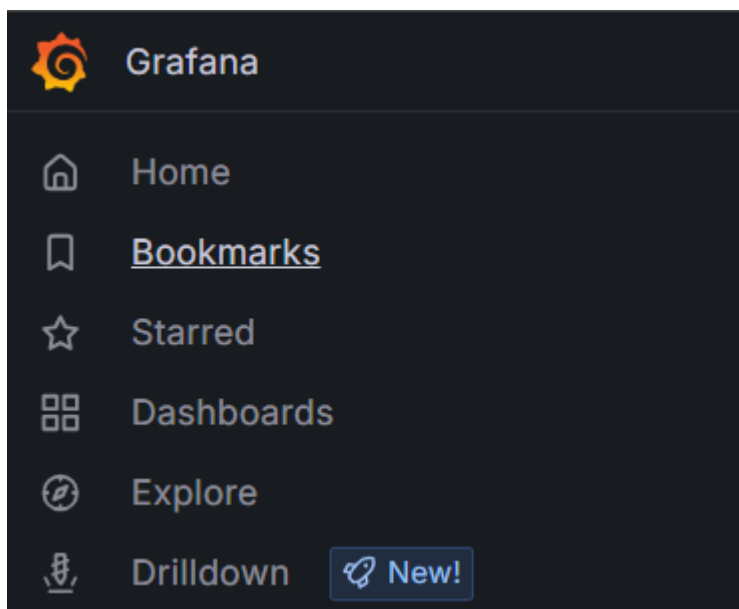


Lo llamamos como queramos y en la conexion ponemos localhost:9090 que es el puerto por defecto en el que se ejecuta prometheus

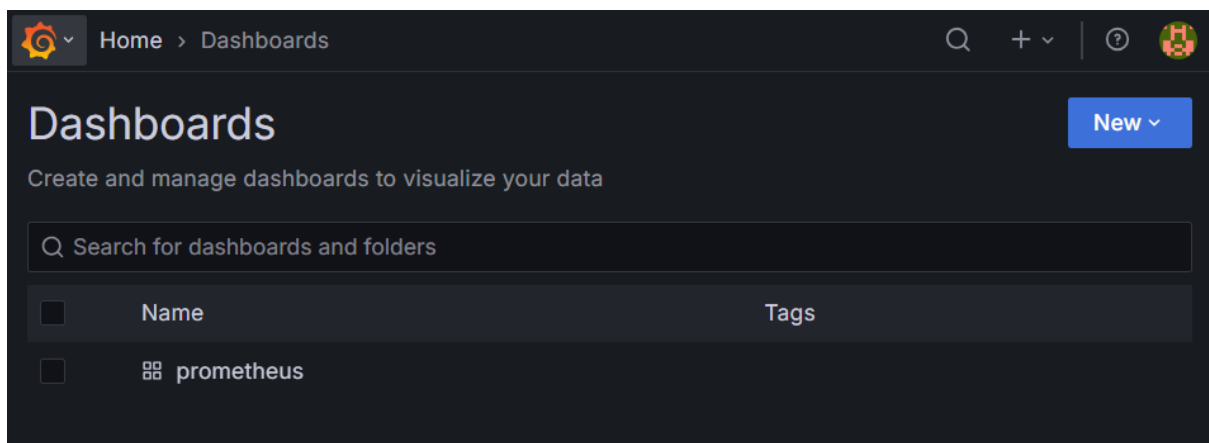


Despues de hacer esto tendemos que ir abajo del todo y darle a save y test para ver que todo esta correctamente

Parte 4: Visualización de métricas en Grafana

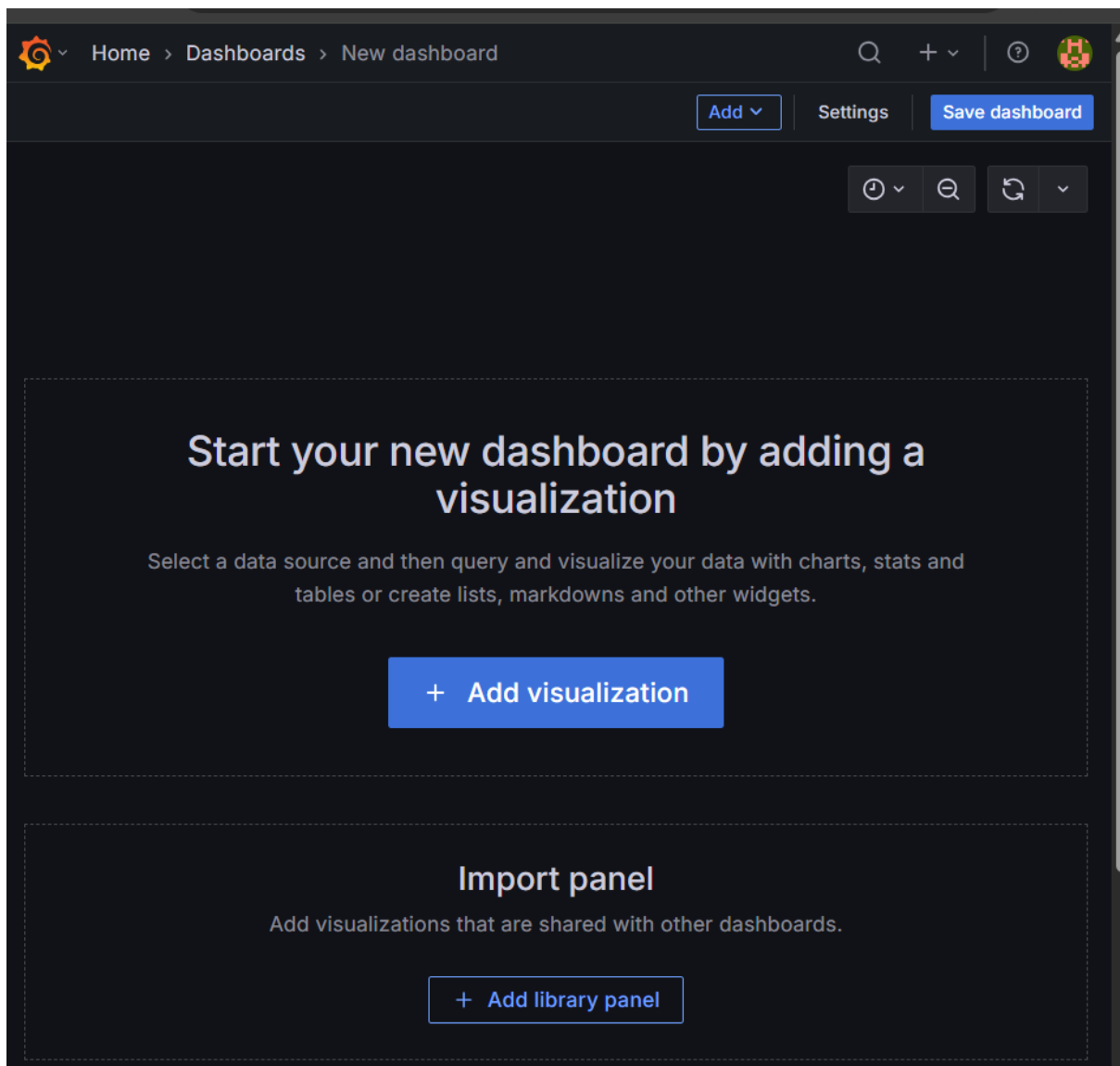


Ahora nos vamos a Dashboards

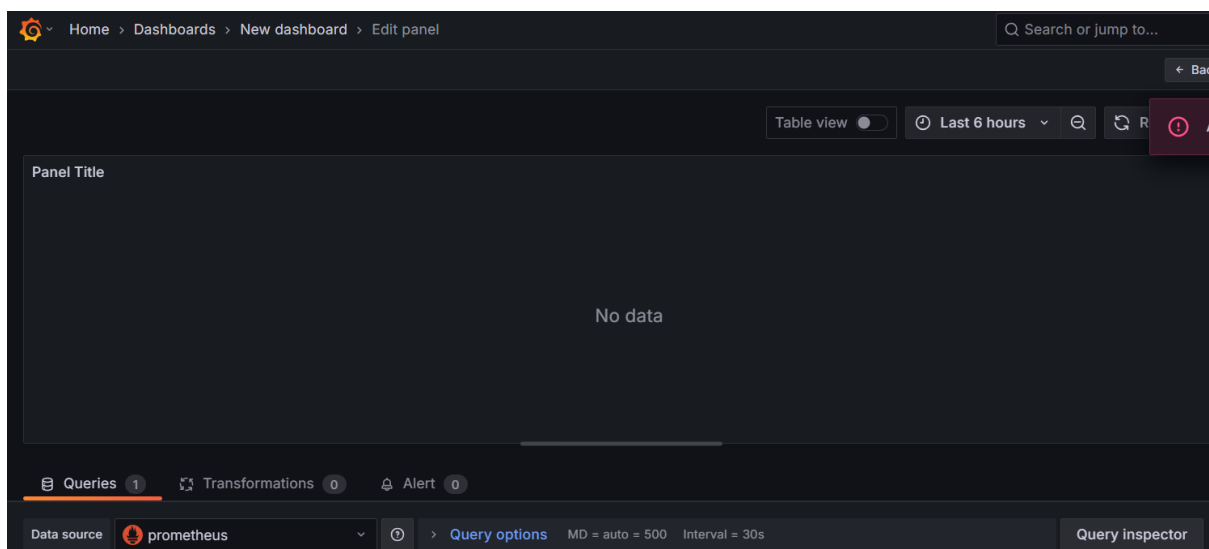


Y creamos un nuevo dashboard

Una vez

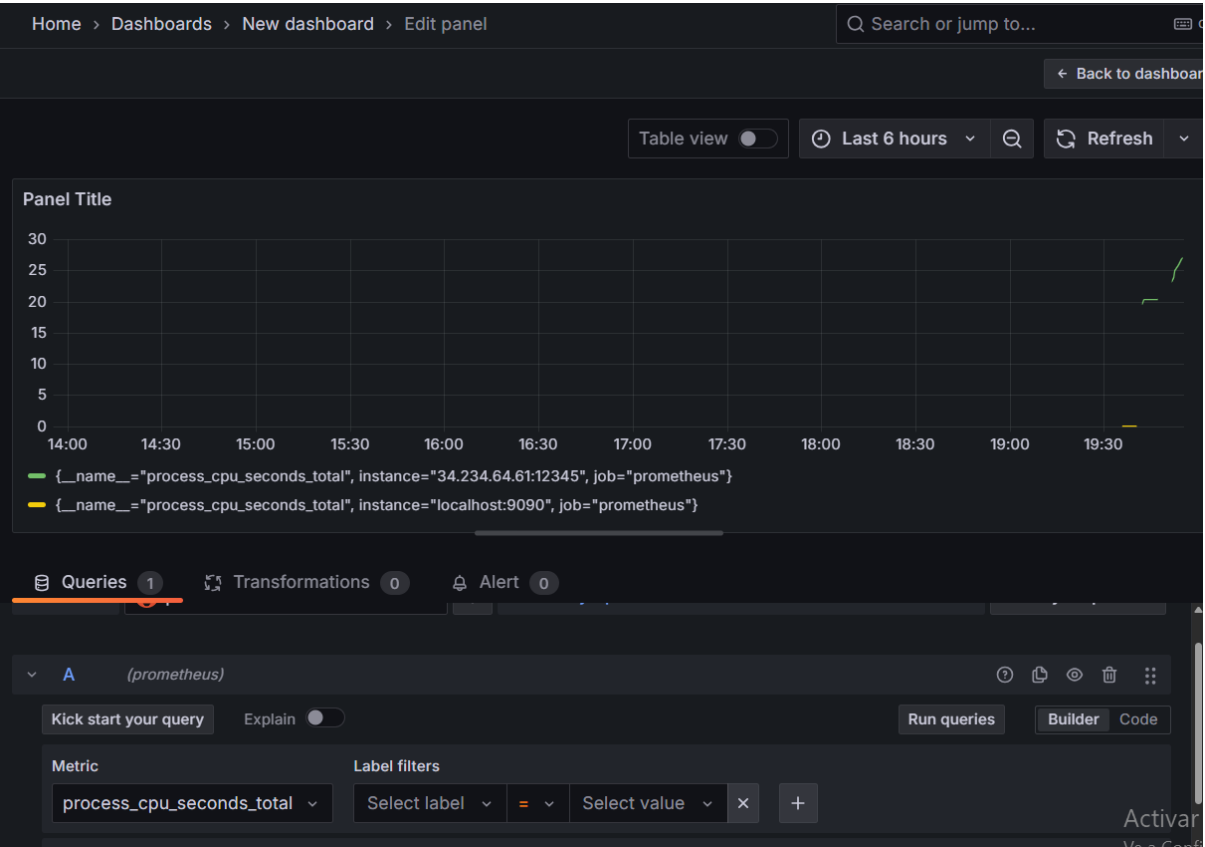


Una vez dado a new dashboard no saldra esto le tenemos que dar a add visualization

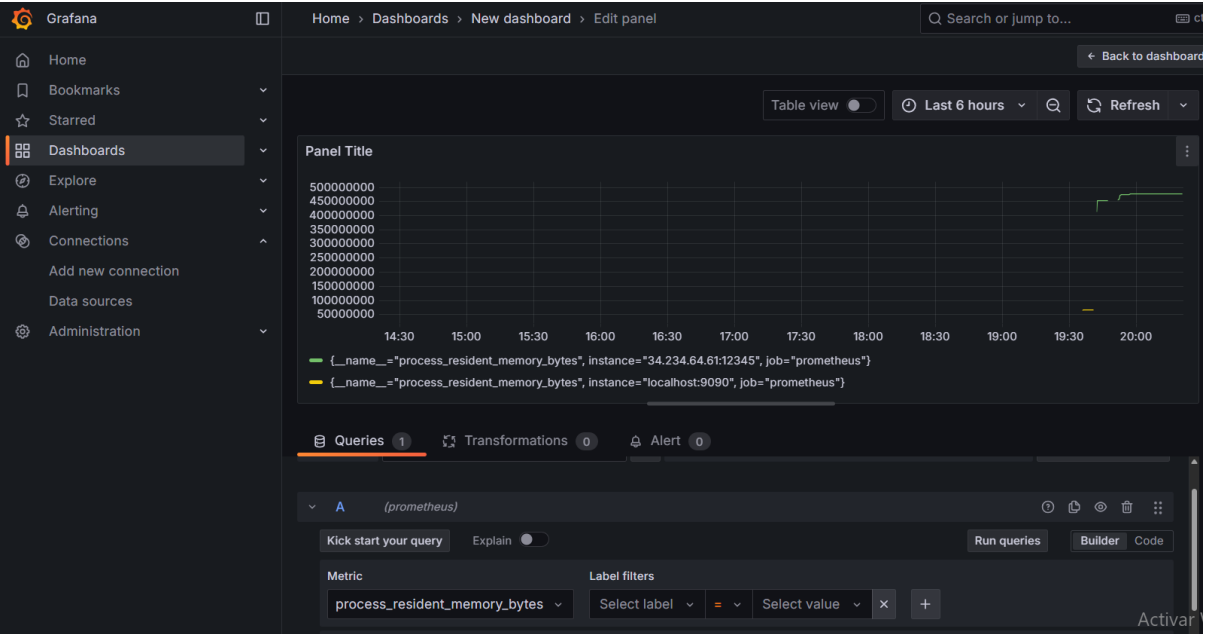


Y debemos tener en el data source el prometheus para que nos salgan las metricas

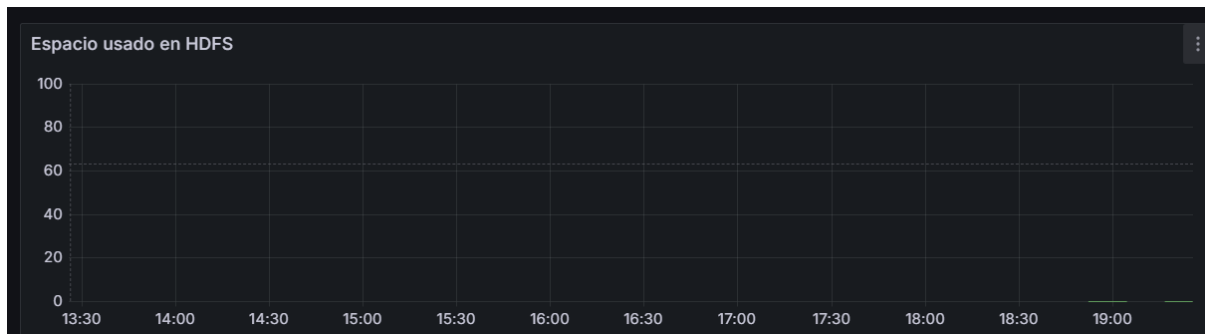
Metrica del Panel de Uso de CPU



Metrica del Panel de Uso de RAM



Metrica del Panel de Espacio en HDFS



Metrica del Estado del NameNode



Preguntas para reflexión

1. ¿Qué métricas consideras más importantes para monitorear en un clúster EMR? ¿Por qué?

Las métricas más importantes para monitorear en un clúster EMR incluyen:

- **Uso de CPU:** Monitorear la utilización de la CPU es crucial para asegurar que los nodos no estén sobrecargados, lo que podría llevar a cuellos de botella en el procesamiento de datos.
- **Memoria:** La memoria utilizada por las aplicaciones es esencial para identificar posibles problemas de rendimiento, como el agotamiento de memoria, que podría hacer que los trabajos se ralenticen o fallen.
- **Estado de los nodos:** La salud de los nodos en el clúster es fundamental para detectar fallos o problemas de conectividad, evitando tiempos de inactividad inesperados.
- **Tiempo de ejecución de los trabajos:** Medir el tiempo de ejecución de los trabajos en Hadoop, Spark o Hive puede ayudar a identificar trabajos que tarden demasiado, indicando posibles ineficiencias.

- **Uso de disco:** Es importante monitorear la capacidad del disco para evitar problemas de almacenamiento, que podrían causar errores de escritura y fallos en los procesos.

Estas métricas son esenciales porque proporcionan una visión integral del estado de los recursos del clúster y ayudan a tomar decisiones informadas sobre escalabilidad y ajustes de rendimiento.

2. ¿Cómo podrías mejorar la configuración de JMX Exporter para recopilar métricas más específicas?

Para mejorar la configuración de JMX Exporter y recopilar métricas más específicas, se pueden tomar las siguientes acciones:

- **Filtrado de métricas relevantes:** En lugar de recopilar todas las métricas de JMX, es posible definir un conjunto de métricas específicas que realmente aporten valor. Esto se puede hacer editando el archivo `config.yml` de JMX Exporter, añadiendo reglas de patrones más específicas para las métricas que se deseen monitorear.
- **Configuración de alertas:** Puedes configurar alertas para determinadas métricas que excedan umbrales definidos, como el uso excesivo de memoria o CPU. Esto permite tomar acciones antes de que ocurran fallos.
- **Agrupar métricas por aplicación:** Si el clúster ejecuta varias aplicaciones (como Hadoop, Spark, Hive), se pueden configurar métricas específicas para cada uno de esos servicios, lo que permite realizar un monitoreo más granular.
- **Ajuste del intervalo de recolección:** Dependiendo de las necesidades del sistema, se puede ajustar el intervalo en el que JMX Exporter recopila las métricas. Recoger datos con mayor frecuencia podría proporcionar más detalles en tiempo real, pero también podría generar una mayor carga sobre los nodos.

3. ¿Qué ventajas tiene usar Prometheus y Grafana frente a otras herramientas de monitoreo?

Las ventajas de usar **Prometheus** y **Grafana** frente a otras herramientas de monitoreo son las siguientes:

- **Prometheus:**
 - **Modelado de datos basado en series temporales:** Prometheus almacena los datos en series temporales, lo que es ideal para el

monitoreo de infraestructuras dinámicas, como un clúster EMR. Esto permite realizar consultas y obtener métricas históricas fácilmente.

- **Escalabilidad y eficiencia:** Prometheus es muy eficiente en términos de almacenamiento y consultas, lo que permite manejar grandes volúmenes de métricas sin degradar el rendimiento.
- **Alertas integradas:** Prometheus incluye un sistema de alertas muy flexible, que puede ser configurado para avisar a los administradores de sistemas en cuanto se detecten problemas, lo que ayuda a una resolución rápida de incidentes.
- **Grafana:**
 - **Visualización de métricas:** Grafana es una de las herramientas de visualización más potentes y flexibles. Ofrece una interfaz muy intuitiva y rica en funcionalidades para crear paneles interactivos con gráficos y métricas en tiempo real.
 - **Integración con múltiples fuentes de datos:** Aunque se suele usar con Prometheus, Grafana puede integrarse con una amplia variedad de otras fuentes de datos (como MySQL, Elasticsearch, etc.), lo que le da mucha flexibilidad.
 - **Paneles personalizables:** Los usuarios pueden crear paneles personalizados con múltiples gráficos y tablas, lo que permite monitorear múltiples métricas en un solo lugar.

En comparación con otras herramientas de monitoreo, como **Zabbix** o **Nagios**, **Prometheus** y **Grafana** ofrecen un enfoque más moderno, flexible y orientado a la nube. Estas herramientas son más fáciles de configurar para entornos distribuidos y proporcionan una experiencia más fluida y visual al manejar grandes volúmenes de métricas en tiempo real.